1. Proposal

A. Introduction

1. Detailed description of idea:

The proposed repository system is intended as a comprehensive library management system for WK_Library. The system will streamline the book borrowing and returning process and facilitate efficient management of library resources and user records.

2. Type of data stored:
The database will store various types of data, including:

Information

The library, such as its name, location, contact details (phone, email).
Student data, including unique identification number (Student_ID), name, grades, and contact information (phone, email).
Employee data, including unique identification number (Staff_ID), name, title, and contact information (phone, email).
Book details such as unique identifier (Book_ID), title, author, publisher, and available quantity.
Available data types include VARCHAR, NUMBER.

A record of transactions, including a unique identifier (Transactions_ID), references to the books, students, and staff involved in the transaction, transaction type, and timestamps of borrow and return dates.
This table's data types have numbers, timestamps.

3. Who will use the database system?
The database system will be mainly used for:
Library staff are responsible for managing book transactions, maintaining

inventory, and updating student and faculty records.
Students borrowing and returning books from the library.

4. How do they use the database system?
Library staff: They will use the system to:

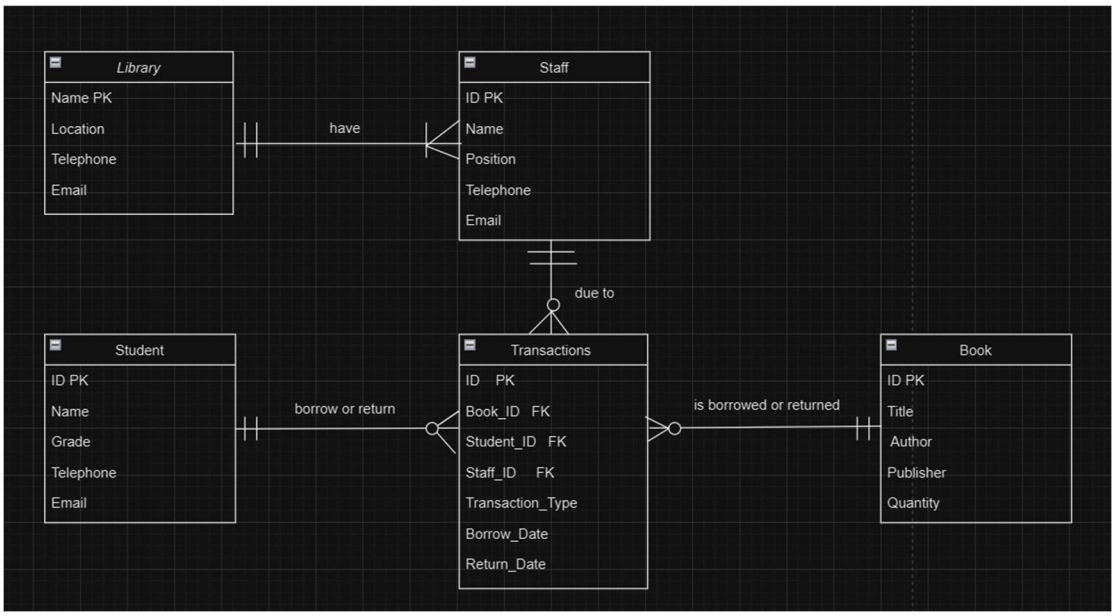
Add new books to inventory.
Keep records of student borrowing and returning books.
Update student and staff records as needed.
Generate reports on book transactions and inventory status.
Students: They will use the system to:

Borrow books by providing your student ID and selecting the books you want.
Return the book by providing the necessary details, such as the book ID or title.

B. Business rules:

There are few assumptions. first, we only use WK_Library and only serve the students from WK campus. Also, one student only borrows or return one book by every transaction. The student can borrow same book many times unlit the book quantity is 0.
A student can borrow multiple books, but each book can be borrowed by multiple students.
Library staff facilitate the borrowing and returning process.
Each book must have a unique identification number (Book_ID).
Each student and staff member must have a unique identification number (Student_ID and Staff_ID).
Transactions must be recorded using a unique identifier (Transactions_ID).

C. Concept data modeling:



The database system should satisfy all relationship types:

One library can be the location for many staff. Therefore, the relationship between library and staff is 1:M.

One staff member is due to complete many tasks of transaction or don't need complete any tasks. The relationship between staff and transaction is 1:M.

One student borrows or returns many books or don't need to borrow any book. Therefore, the relationship between staff and transaction is 1:M.

One book is borrowed or returned by many students. So, the transaction may be a lot of or nothing. Therefore, the relationship between Book and transaction is 1:M.

2. Implementation Details

Create table:

```
DROP TABLE Library;
DROP TABLE Student;
DROP TABLE Staff;
DROP TABLE Book;

-- Create Library Table
CREATE TABLE Library (
    Library_Name VARCHAR(20),
    Location VARCHAR(20),
    Telephone NUMBER(10),
    E_mail VARCHAR(50),
    CONSTRAINT Library_ID_pk PRIMARY KEY (Library_Name)
);

-- Create Student Table
CREATE TABLE Student (
    Student_ID NUMBER(20),
    Student_Name VARCHAR(20),
    Grade NUMBER(2),
    Telephone NUMBER(10),
    E_mail VARCHAR(50),
    CONSTRAINT Student_ID_pk PRIMARY KEY (Student_ID)
);

-- Create Staff Table
CREATE TABLE Staff (
    Staff_ID NUMBER(3),
    Staff_Name VARCHAR(20),
    Position VARCHAR(20),
    Telephone NUMBER(10),
    E_mail VARCHAR(50),
    CONSTRAINT Staff_ID_pk PRIMARY KEY (Staff_ID)
);

-- Create Book Table
CREATE TABLE Book (
    Book_ID NUMBER(4),
    Title VARCHAR(200),
    Author VARCHAR(50),
    Publisher VARCHAR(20),
    Quantity NUMBER(3),
    CONSTRAINT Book_ID_pk PRIMARY KEY (Book_ID)
);
```

```
-- Create Transactions Table
CREATE TABLE Transactions (
    Transactions_ID NUMBER(20),
    Book_ID NUMBER(4),
    Student_ID NUMBER(3),
    Staff_ID NUMBER(3),
    Transaction_Type VARCHAR(10),
    Borrow_Date TIMESTAMP,
    Return_Date TIMESTAMP,

    CONSTRAINT Transactions_ID_pk PRIMARY KEY (Transactions_ID),
    CONSTRAINT Transactions_fk_Book FOREIGN KEY (Book_ID) REFERENCES Book(Book_ID),
    CONSTRAINT Transactions_fk_Student FOREIGN KEY (Student_ID) REFERENCES Student(Student_ID),
    CONSTRAINT Transactions_fk_Staff FOREIGN KEY (Staff_ID) REFERENCES Staff(Staff_ID)
);
```

Table created.

Table created.

Table created.

Table created.

Table created.

TABLE LIBRARY

Column	Null?	Type
LIBRARY_NAME	NOT NULL	VARCHAR2(20)
LOCATION	-	VARCHAR2(20)
TELEPHONE	-	NUMBER(10,0)
E_MAIL	-	VARCHAR2(50)

4 rows selected.

TABLE STUDENT

Column	Null?	Type
STUDENT_ID	NOT NULL	NUMBER(20,0)
STUDENT_NAME	-	VARCHAR2(20)
GRADE	-	NUMBER(2,0)
TELEPHONE	-	NUMBER(10,0)
E_MAIL	-	VARCHAR2(50)

Download CSV

5 rows selected.

TABLE BOOK

Column	Null?	Type
BOOK_ID	NOT NULL	NUMBER(4,0)
TITLE	-	VARCHAR2(200)
AUTHOR	-	VARCHAR2(50)
PUBLISHER	-	VARCHAR2(20)
QUANTITY	-	NUMBER(3,0)

TABLE TRANSACTIONS

Column	Null?	Type
TRANSACTIONS_ID	NOT NULL	NUMBER(20,0)
BOOK_ID	-	NUMBER(4,0)
STUDENT_ID	-	NUMBER(3,0)
STAFF_ID	-	NUMBER(3,0)
TRANSACTION_TYPE	-	VARCHAR2(10)
BORROW_DATE	-	TIMESTAMP(6)
RETURN_DATE	-	TIMESTAMP(6)

Insert value:

```
INSERT INTO Library ( Library_Name, Location, Telephone, E_mail )VALUES ( 'WK_Library', ' West Kowloon',12345678,'123@gmail.com' );
INSERT INTO Library ( Library_Name, Location, Telephone, E_mail )VALUES ('HHB_Library', 'Hung HOM Bay',12345679,'124@gmail.com');
INSERT INTO Library ( Library_Name, Location, Telephone, E_mail )VALUES ('TSW_Library','Tin Shui Wai',12345670,'125@gmail.com');
INSERT INTO Library ( Library_Name, Location, Telephone, E_mail )VALUES ('TM_Library','Tuen Mun',12345680,'126@gmail.com');
INSERT INTO Library ( Library_Name, Location, Telephone, E_mail )VALUES ('ST_Library','Sha Tin',12345666,'127@gmail.com');


INSERT INTO Student (Student_ID, Student_Name, Grade , Telephone, E_mail )VALUES (100, 'Mary' ,1,34567812,'543@gmail.com' );
INSERT INTO Student (Student_ID, Student_Name, Grade , Telephone, E_mail )VALUES (101, 'May' ,3,34567813,'542@gmail.com' );
INSERT INTO Student (Student_ID, Student_Name, Grade , Telephone, E_mail )VALUES (102, 'Malloy' ,4,34567814,'541@gmail.com' );
INSERT INTO Student (Student_ID, Student_Name, Grade , Telephone, E_mail )VALUES (103, 'Maria' ,2,34567815,'544@gmail.com' );
INSERT INTO Student (Student_ID, Student_Name, Grade , Telephone, E_mail )VALUES (104, 'Macy' ,1,34567816,'545@gmail.com' );

INSERT INTO Staff (Staff_ID, Staff_Name, Position , Telephone, E_mail )VALUES (10, 'Tom' ,'Chief Librarian',87654321,'321@gmail.com' );
INSERT INTO Staff (Staff_ID, Staff_Name, Position , Telephone, E_mail )VALUES (11,'Tim', 'Assistant Librarian',87654322,'322@gmail.com');
INSERT INTO Staff (Staff_ID, Staff_Name, Position , Telephone, E_mail )VALUES (12,'Tommy','Library Assistant',87654323,'323@gmail.com');
INSERT INTO Staff (Staff_ID, Staff_Name, Position , Telephone, E_mail )VALUES (13,'Tony','Library Assistant',87654324,'324@gmail.com');
INSERT INTO Staff (Staff_ID, Staff_Name, Position , Telephone, E_mail )VALUES (14,'Tristian','Library Assistant',87654325,'325@gmail.com');

INSERT INTO Book (Book_ID, Title, Author, Publisher,  Quantity)VALUES (1001, 'International Business: Concepts, Cases and Exercises' ,'Frank F.Cotae','Routledge',10 );
INSERT INTO Book (Book_ID, Title, Author, Publisher,  Quantity)VALUES (1002, "Policy Solutions for Economic Growth in a Developing Country: Perspectives on Afghanistan's Trade and I
INSERT INTO Book (Book_ID, Title, Author, Publisher,  Quantity)VALUES (1003, 'Mastering Screenplay Form and Style' ,'Mick Hurbis-Cherrier','Focal Pr',10 );
INSERT INTO Book (Book_ID, Title, Author, Publisher,  Quantity)VALUES (1004, 'My English Language Journey' ,'Ghassoub Sharif Mustafa','Austin Macauley',10 );
INSERT INTO Book (Book_ID, Title, Author, Publisher,  Quantity)VALUES (1005, 'Unexpected: The Autobiography' ,'Greg Rutherford','Simon & Schuster Ltd',10 );

select * from Library;
select * from Student;
select * from Staff;
select * from Book;
```

LIBRARY_NAME	LOCATION	TELEPHONE	E_MAIL
WK_Library	West Kowloon	12345678	123@gmail.com
HHB_Library	Hung HOM Bay	12345679	124@gmail.com
TSW_Library	Tin Shui Wai	12345670	125@gmail.com
TM_Library	Tuen Mun	12345680	126@gmail.com
ST_Library	Sha Tin	12345666	127@gmail.com

Download CSV

5 rows selected.

STUDENT_ID	STUDENT_NAME	GRADE	TELEPHONE	E_MAIL
100	Mary	1	34567812	543@gmail.com
101	May	3	34567813	542@gmail.com
102	Malloy	4	34567814	541@gmail.com
103	Maria	2	34567815	544@gmail.com
104	Macy	1	34567816	545@gmail.com

STAFF_ID	STAFF_NAME	POSITION	TELEPHONE	E_MAIL
10	Tom	Chief Librarian	87654321	321@gmail.com
11	Tim	Assistant Librarian	87654322	322@gmail.com
12	Tommy	Library Assistant	87654323	323@gmail.com
13	Tony	Library Assistant	87654324	324@gmail.com
14	Tristian	Library Assistant	87654325	325@gmail.com

Download CSV

5 rows selected.

BOOK_ID	TITLE	AUTHOR	PUBLISHER	QUANTITY
1001	International Business: Concepts, Cases and Exercises	Frank F.Cotae	Routledge	10
1003	Mastering Screenplay Form and Style	Mick Hurbis-Cherrier	Focal Pr	10
1004	My English Language Journey	Ghassoub Sharif Mustafa	Austin Macauley	9
1005	Unexpected: The Autobiography	Greg Rutherford	Simon & Schuster Ltd	10

Trigger:

```
98  -- Create the trigger
99  CREATE OR REPLACE TRIGGER enforce_quantity_constraint
100 BEFORE INSERT ON transactions
101 FOR EACH ROW
102 DECLARE
103     v_current_quantity NUMBER;
104 BEGIN
105     -- Get the current quantity of the book
106     SELECT Quantity INTO v_current_quantity
107     FROM Book
108     WHERE Book_ID = :NEW.Book_ID;
109
110     -- Check if the transaction is a borrow operation
111     IF :NEW.Transaction_Type = 'BORROW' THEN
112         -- Ensure there are enough available copies to borrow
113         IF v_current_quantity <= 0 THEN
114             RAISE_APPLICATION_ERROR(-20001, 'No available copies of this book.');
115         END IF;
116     END IF;
117 END;
118 /
```

When the quantity is <=0. Then the book cannot checkout again.

For example:

TRANSACTIONS_ID	BOOK_ID	STUDENT_ID	STAFF_ID	TRANSACTION_TYPE	BORROW_DATE	RETURN_DATE
2	1004	100	10	BORROW	24-MAR-24 10.21.36.141846 AM	-
10	1004	100	10	BORROW	24-MAR-24 10.31.45.183880 AM	-
1	1004	100	10	BORROW	24-MAR-24 10.14.43.320411 AM	-
3	1004	100	10	BORROW	24-MAR-24 10.30.43.504362 AM	-
4	1004	100	10	BORROW	24-MAR-24 10.30.52.406448 AM	-
5	1004	100	10	BORROW	24-MAR-24 10.30.54.675720 AM	-
6	1004	100	10	BORROW	24-MAR-24 10.31.02.536118 AM	-
7	1004	100	10	BORROW	24-MAR-24 10.31.07.087643 AM	-
8	1004	100	10	BORROW	24-MAR-24 10.31.29.077295 AM	-
9	1004	100	10	BORROW	24-MAR-24 10.31.37.765326 AM	-

BOOK_ID	TITLE	AUTHOR	PUBLISHER	QUANTITY
1001	International Business: Concepts, Cases and Exercises	Frank F.Cotae	Routledge	10
1003	Mastering Screenplay Form and Style	Mick Hurbis-Cherrier	Focal Pr	10
1004	My English Language Journey	Ghassoub Sharif Mustafa	Austin Macauley	0
1005	Unexpected: The Autobiography	Greg Rutherford	Simon & Schuster Ltd	10

Then it will show you the error messages.

```
Statement processed.
Error borrowing book: ORA-20001: No available copies of this book.
```

Procedure:

1.Borrow Book

```
PROCEDURE borrow_book(
  p_book_id IN NUMBER,
  p_student_id IN NUMBER,
  p_staff_id IN NUMBER
) AS
BEGIN

  -- Insert a new transaction record
  INSERT INTO transactions (Transactions_ID, Book_ID, Student_ID, Staff_ID, Transaction_Type, Borrow_Date, Return_Date)
  VALUES (transaction_seq.NEXTVAL, p_book_id, p_student_id, p_staff_id, 'BORROW', SYSTIMESTAMP, NULL);

   -- Decrease the quantity of the book
  UPDATE Book
  SET Quantity = Quantity - 1
  WHERE Book_id = p_book_id;


  COMMIT;
EXCEPTION
  WHEN OTHERS THEN
    ROLLBACK;
    RAISE;
END borrow_book;
```

When someone borrows a book. The value of data will insert to transaction table about Student ID, transaction Type and Borrow Date. And the quantity of the book will reduce until to 0.

For example:

TRANSACTIONS_ID	BOOK_ID	STUDENT_ID	STAFF_ID	TRANSACTION_TYPE	BORROW_DATE	RETURN_DATE
2	1004	100	10	BORROW	24-MAR-24 10.21.36.141846 AM	-
10	1004	100	10	BORROW	24-MAR-24 10.31.45.183880 AM	-
1	1004	100	10	BORROW	24-MAR-24 10.14.43.320411 AM	-
3	1004	100	10	BORROW	24-MAR-24 10.30.43.504362 AM	-
4	1004	100	10	BORROW	24-MAR-24 10.30.52.406448 AM	-
5	1004	100	10	BORROW	24-MAR-24 10.30.54.675720 AM	-
6	1004	100	10	BORROW	24-MAR-24 10.31.02.536118 AM	-
7	1004	100	10	BORROW	24-MAR-24 10.31.07.087643 AM	-
8	1004	100	10	BORROW	24-MAR-24 10.31.29.077295 AM	-
9	1004	100	10	BORROW	24-MAR-24 10.31.37.765326 AM	-

2. Return Book

```
PROCEDURE borrow_book(
  p_book_id IN NUMBER,
  p_student_id IN NUMBER,
  p_staff_id IN NUMBER
) AS
BEGIN

  -- Insert a new transaction record
  INSERT INTO transactions (Transactions_ID, Book_ID, Student_ID, Staff_ID, Transaction_Type, Borrow_Date, Return_Date)
  VALUES (transaction_seq.NEXTVAL, p_book_id, p_student_id, p_staff_id, 'BORROW', SYSTIMESTAMP, NULL);

   -- Decrease the quantity of the book
  UPDATE Book
  SET Quantity = Quantity - 1
  WHERE Book_id = p_book_id;


  COMMIT;
EXCEPTION
  WHEN OTHERS THEN
    ROLLBACK;
    RAISE;
END borrow_book;
```

When someone returns the book. The value of data will update to transaction table about Return Date, transaction data type and, the quantity of the book.

For example:

BOOK_ID	TITLE	AUTHOR	PUBLISHER	QUANTITY
1001	International Business: Concepts, Cases and Exercises	Frank F.Cotae	Routledge	10
1003	Mastering Screenplay Form and Style	Mick Hurbis-Cherrier	Focal Pr	10
1004	My English Language Journey	Ghassoub Sharif Mustafa	Austin Macauley	1
1005	Unexpected: The Autobiography	Greg Rutherford	Simon & Schuster Ltd	10

TRANSACTIONS_ID	BOOK_ID	STUDENT_ID	STAFF_ID	TRANSACTION_TYPE	BORROW_DATE	RETURN_DATE
2	1004	100	10	RETURN	24-MAR-24 10.21.36.141846 AM	24-MAR-24 10.49.11.105835 AM
10	1004	100	10	RETURN	24-MAR-24 10.31.45.183880 AM	24-MAR-24 10.51.33.827932 AM
1	1004	100	10	BORROW	24-MAR-24 10.14.43.320411 AM	-
3	1004	100	10	BORROW	24-MAR-24 10.30.43.504362 AM	-
4	1004	100	10	BORROW	24-MAR-24 10.30.52.406448 AM	-
5	1004	100	10	BORROW	24-MAR-24 10.30.54.675720 AM	-
6	1004	100	10	BORROW	24-MAR-24 10.31.02.536118 AM	-
7	1004	100	10	BORROW	24-MAR-24 10.31.07.087643 AM	-
8	1004	100	10	BORROW	24-MAR-24 10.31.29.077295 AM	-
9	1004	100	10	BORROW	24-MAR-24 10.31.37.765326 AM	-

Function:

Get available book and get borrowed book.

```
FUNCTION get_available_books RETURN SYS_REFCURSOR AS
  c SYS_REFCURSOR;
BEGIN
  OPEN c FOR
  SELECT *
  FROM Book
  WHERE Quantity > 0;

  RETURN c;
EXCEPTION
  WHEN OTHERS THEN
    ROLLBACK;
    RAISE;
END get_available_books;

FUNCTION get_borrowed_books(
  p_student_id IN NUMBER
) RETURN SYS_REFCURSOR AS
  c SYS_REFCURSOR;
BEGIN
  OPEN c FOR
  SELECT b.*
  FROM Book b
  INNER JOIN Transactions t ON b.Book_ID = t.Book_ID
  WHERE t.Student_ID = p_student_id;

  RETURN c;
EXCEPTION
  WHEN OTHERS THEN
    ROLLBACK;
    RAISE;
END get_borrowed_books;
```

Function of get_available_book is checking quantity of the book from Book table. If the quantity >0, it is available to borrow to student.

Function of get_borrowed_books is getting the value about the book and student to procedure.

Query:

1. Check Out

```
-- Checkout Book
DECLARE
    v_student_id NUMBER;
    v_book_id NUMBER;
    v_staff_id NUMBER;
BEGIN
    -- Retrieve the IDs of the student, book, and staff
    SELECT Student_id INTO v_student_id FROM Student WHERE Student_name = 'Mary';
    SELECT Book_id INTO v_book_id FROM Book WHERE Title = 'My English Language Journey';
    SELECT Staff_id INTO v_staff_id FROM Staff WHERE staff_name = 'Tom';

    -- Execute the borrow_book procedure
    library_system.borrow_book(p_book_id => v_book_id, p_student_id => v_student_id, p_staff_id => v_staff_id);

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Book borrowed successfully by Mary from staff Tom.');
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error borrowing book: ' || SQLERRM);
END;
/
```

Mary (Stundent_ID: 100) borrows the book called ‘My English Language Journey’ (Book_ID: 1004) and the staff Tom (Staff_ID: 10) due to complete the transaction of check out.

The result:

TRANSACTIONS_ID	BOOK_ID	STUDENT_ID	STAFF_ID	TRANSACTION_TYPE	BORROW_DATE	RETURN_DATE
2	1004	100	10	BORROW	24-MAR-24 10.21.36.141846 AM	-
10	1004	100	10	BORROW	24-MAR-24 10.31.45.183880 AM	-
1	1004	100	10	BORROW	24-MAR-24 10.14.43.320411 AM	-
3	1004	100	10	BORROW	24-MAR-24 10.30.43.504362 AM	-
4	1004	100	10	BORROW	24-MAR-24 10.30.52.406448 AM	-
5	1004	100	10	BORROW	24-MAR-24 10.30.54.675720 AM	-
6	1004	100	10	BORROW	24-MAR-24 10.31.02.536118 AM	-
7	1004	100	10	BORROW	24-MAR-24 10.31.07.087643 AM	-
8	1004	100	10	BORROW	24-MAR-24 10.31.29.077295 AM	-
9	1004	100	10	BORROW	24-MAR-24 10.31.37.765326 AM	-

2. Return Book

```
                                                    -- Return Book
DECLARE
    v_transaction_id NUMBER;
BEGIN
    -- Retrieve the transaction ID for the book borrowed by Mary
    SELECT Transactions_ID INTO v_transaction_id
    FROM Transactions t
    JOIN Student s ON t.student_id = s.student_id
    JOIN Book b ON t.book_id = b.book_id
    WHERE s.Student_name = 'Mary'
    AND t.Transaction_Type = 'BORROW'
    AND b.Title = 'My English Language Journey'
    AND ROWNUM = 1; -- Limit the result to one row only

    -- Execute the return_book procedure
    library_system.return_book(p_transaction_id => v_transaction_id);

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Book returned successfully by Mary.');
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No borrowed book found for Mary.');
    WHEN OTHERS THEN
        ROLLBACK;
        DBMS_OUTPUT.PUT_LINE('Error returning book: ' || SQLERRM);
END;
/
```

Mary (Stundent_ID: 100) returns the book called ‘My English Language Journey’ (Book_ID: 1004) and the staff Tom (Staff_ID: 10) due to complete the transaction of check out.

The result:

TRANSACTIONS_ID	BOOK_ID	STUDENT_ID	STAFF_ID	TRANSACTION_TYPE	BORROW_DATE	RETURN_DATE
2	1004	100	10	RETURN	24-MAR-24 10.21.36.141846 AM	24-MAR-24 10.49.11.105835 AM
10	1004	100	10	RETURN	24-MAR-24 10.31.45.183880 AM	24-MAR-24 10.51.33.827932 AM
1	1004	100	10	BORROW	24-MAR-24 10.14.43.320411 AM	-
3	1004	100	10	BORROW	24-MAR-24 10.30.43.504362 AM	-
4	1004	100	10	BORROW	24-MAR-24 10.30.52.406448 AM	-
5	1004	100	10	BORROW	24-MAR-24 10.30.54.675720 AM	-
6	1004	100	10	BORROW	24-MAR-24 10.31.02.536118 AM	-
7	1004	100	10	BORROW	24-MAR-24 10.31.07.087643 AM	-
8	1004	100	10	BORROW	24-MAR-24 10.31.29.077295 AM	-
9	1004	100	10	BORROW	24-MAR-24 10.31.37.765326 AM	-

3. query with all SELECT, FROM, WHERE, GROUPBY, HAVING AND ORDER BY clauses on a table

```
SELECT Student_Name, Grade, COUNT(*) AS Total_Books_Borrowed
FROM Student
JOIN Transactions ON Student.Student_ID = Transactions.Student_ID
WHERE Transaction_Type = 'BORROW'
GROUP BY Student_Name, Grade
HAVING COUNT(*) > 1
ORDER BY Total_Books_Borrowed DESC;
```

Checking the student how many did she borrowed

The result:

STUDENT_NAME	GRADE	TOTAL_BOOKS_BORROWED
Mary	1	8

4. 1 query that joins at least two tables

```
SELECT Transactions.Transactions_ID, Student.Student_Name, Book.Title, Transactions.Borrow_Date
FROM Transactions
JOIN Student ON Transactions.Student_ID = Student.Student_ID
JOIN Book ON Transactions.Book_ID = Book.Book_ID
ORDER BY Transactions.Borrow_Date DESC;
```

Checking the record of borrowed book

The result:

TRANSACTIONS_ID	STUDENT_NAME	TITLE	BORROW_DATE
10	Mary	My English Language Journey	24-MAR-24 10.31.45.183880 AM
9	Mary	My English Language Journey	24-MAR-24 10.31.37.765326 AM
8	Mary	My English Language Journey	24-MAR-24 10.31.29.077295 AM
7	Mary	My English Language Journey	24-MAR-24 10.31.07.087643 AM
6	Mary	My English Language Journey	24-MAR-24 10.31.02.536118 AM
5	Mary	My English Language Journey	24-MAR-24 10.30.54.675720 AM
4	Mary	My English Language Journey	24-MAR-24 10.30.52.406448 AM
3	Mary	My English Language Journey	24-MAR-24 10.30.43.504362 AM
2	Mary	My English Language Journey	24-MAR-24 10.21.36.141846 AM
1	Mary	My English Language Journey	24-MAR-24 10.14.43.320411 AM

5. 1 query that uses a subquery

```
SELECT Student_Name, Grade
FROM Student
WHERE Student_ID IN (
    SELECT DISTINCT Student_ID
    FROM Transactions
    WHERE Transaction_Type = 'BORROW'
);
```

Checking the grade of students who borrowed a book.

STUDENT_NAME	GRADE
Mary	1